

PySCF and Quantum Circuit Generation for VQE/QAOA

PySCF (Python-based Simulations of Chemistry Framework) itself is a classical electronic structure library, but it provides the *building blocks* (molecular integrals, Hamiltonians, etc.) needed to construct quantum circuits for chemistry problems. By combining PySCF with quantum computing frameworks, we can generate many *variational quantum circuits* (e.g. for VQE or related algorithms) and export them to QASM. Below, we outline all mainstream ways to create quantum circuits using PySCF, along with variations to reach a large number of circuits.

1. PySCF + Qiskit Nature (Quantum Chemistry with Qiskit)

Qiskit Nature integrates directly with PySCF to obtain molecular Hamiltonians and build quantum circuits for VQE. A **PySCFDriver** in Qiskit reads the molecule definition (atoms, basis set, charge, etc.) and returns an `ElectronicStructureProblem` ¹ ². From there:

- **Ansatz Circuits:** Qiskit provides standard circuit templates for chemistry, such as the **Hartree-Fock state** preparation and **Unitary Coupled-Cluster (UCC) ansatz**. For example, one can create a UCCSD ansatz circuit with a Hartree-Fock initial state in Qiskit ³. This produces a `QuantumCircuit` representing the variational state $|\psi(\theta)\rangle = U(\theta)|\text{HF}\rangle$. The UCCSD ansatz is a chemically-inspired circuit (parameterized excitations) widely used as a VQE benchmark.
- **Hardware-Efficient Circuits:** Qiskit also offers hardware-efficient ansätze (e.g. EfficientSU2 or custom layered circuits). These are not chemistry-specific but can be applied to the qubits corresponding to molecular orbitals. For instance, an IBM tutorial modeled a **LUCJ (Local Unitary Cluster Jastrow)** ansatz for N_2 , which includes layers of two-qubit rotations optimized for hardware ⁴ ⁵. Hardware-efficient ansätze (like the one from Kandala *et al.*, 2017) are another source of variational circuits, often with entangling layers of CZ or CNOT gates alternating with single-qubit rotations ⁶ ⁷.
- **QASM Export:** Circuits constructed in Qiskit can be directly exported to OpenQASM. Qiskit's `QuantumCircuit.qasm()` method outputs OpenQASM 2.0 by default, and Qiskit now also supports OpenQASM 3 (via `qasm3` dumps) ⁸. In practice, this means any circuit from the Qiskit-PySCF workflow (UCC, hardware-efficient, etc.) can be saved as a QASM file. For example, one can use Qiskit to build the circuit and then do `qc.qasm()` or `qasm3.dumps(qc)` to get a QASM string ⁹ ¹⁰. This ensures the circuits are "QASM-compatible" for your dataset.

Variation: By using Qiskit + PySCF, you can generate many circuits by varying the molecule and ansatz: e.g. UCCSD circuits for different molecules (H_2 , LiH , H_2O , NH_3 , etc. each in a minimal basis) and with different qubit mappings (Jordan-Wigner vs. Bravyi-Kitaev via Qiskit's mappers ¹¹). Each unique combination yields

a distinct quantum circuit. You can also adjust ansatz depth or include/exclude certain excitations (Qiskit Nature has variants like PUCCD, SUCCD, etc. ¹²) to get even more circuits.

2. PySCF + OpenFermion (with Cirq or Others)

OpenFermion is an open-source toolkit for quantum chemistry that can use PySCF as a backend (via **OpenFermion-PySCF** plugin) to compute molecular integrals. With OpenFermion, one can generate qubit Hamiltonians and then construct circuits for algorithms like VQE or quantum simulation:

- **Hamiltonian to Qubit Operator:** Using PySCF, OpenFermion can output the molecular Hamiltonian in second-quantized form and then map it to a qubit operator (e.g. via Jordan-Wigner transformation). For instance, OpenFermion's `MolecularData` with `run_pyscf` will give you a Fermion operator which you convert to a `QubitOperator` ¹³.
- **Ansatz Circuits (UCC and More):** OpenFermion provides functions to generate UCCSD excitation operators and can interface with circuit libraries. One approach is using **OpenFermion-Cirq**: it has utilities to convert fermionic operators into quantum circuits. The *Google Cirq* framework can then simulate or output these circuits. As an example, after obtaining the Hamiltonian for H_2 with PySCF, one can use OpenFermion+Cirq to build an ansatz or even trotterize the Hamiltonian. In one workflow, Qooba *et al.* show that you can compile a molecule's Hamiltonian into an optimized quantum circuit using OpenFermion and Cirq ¹⁴ ¹⁵. They list methods like basis-change circuits and Trotter steps to create the circuit, resulting in a specific 4-qubit circuit for H_2 ¹⁶.
- **Trotter and QPE Circuits:** Beyond VQE, OpenFermion allows constructing circuits for Hamiltonian simulation. By trotterizing the molecular Hamiltonian, you get circuits that approximate e^{-iHt} . While these are more relevant for Quantum Phase Estimation, they can serve as additional circuit instances. OpenFermion's documentation mentions outputting circuits in QASM format for quantum simulation experiments ¹⁷. For example, OpenFermion can produce an OpenQASM circuit for a chemistry Hamiltonian that could be used as a benchmark circuit ¹⁷.
- **Integration with Other SDKs:** OpenFermion outputs can be fed into other libraries too. You could take the qubit Hamiltonian from OpenFermion-PySCF and use **Qiskit's VQE** (as in Qiskit's tutorials) or **Cirq's ansatz classes**. Cirq can export circuits to QASM as well (Cirq has `cirq.qasm()` for OpenQASM 2.0 support).

Variation: By choosing different molecules and different circuit construction methods in OpenFermion, you can populate your dataset with many circuits. For instance, you might generate: UCCSD circuits via OpenFermion for a range of molecules; hardware-efficient circuits (if you manually design them) for those same Hamiltonians; and even random trotterized circuits for various chemistry Hamiltonians. Each choice (molecule, ansatz type, simulation time/steps for trotter, etc.) produces a new circuit instance.

3. PySCF + PennyLane (PennyLane QChem Module)

PennyLane's `qml.qchem` module can use PySCF as a backend to set up quantum chemistry problems and then leverage PennyLane's parametric circuit capabilities to create VQE circuits. Key points:

- **Hamiltonian Generation:** PennyLane can compute molecular Hamiltonians by calling PySCF under the hood (with `method="pyscf"` or using OpenFermion)¹⁸¹⁹. For example, `qml.qchem.molecular_hamiltonian(molecule, method="pyscf")` returns a qubit Hamiltonian (as a sum of Pauli terms) for the given molecule²⁰. This uses PySCF to solve Hartree-Fock and get integrals, similar to Qiskit's driver²¹.
- **Ansätze and Circuits:** With the Hamiltonian in PennyLane, you can then choose an ansatz. PennyLane provides **parametric circuit templates** – for instance, a hardware-efficient ansatz (layers of RX/RY and CNOTs) or you can implement UCCSD manually. PennyLane's flexibility allows you to define a quantum circuit with gates as a Python function (using `qml.RX`, `qml.CNOT`, etc.). In their VQE demos, PennyLane often uses hardware-efficient ansätze or device-specific templates. (As of 2025, PennyLane does not have a built-in UCCSD *template*, but users can construct UCCSD using the excitation generators from OpenFermion – PennyLane's *differentiable* nature even allows gradient-based optimization of circuit structures²².)
- **Initial States:** PennyLane can also import initial state preparations from PySCF. For example, one can prepare the Hartree-Fock state on the qubits either by using OpenFermion's `jordan_wigner` mapping or PennyLane's `qml.qchem.hf_state` function (which yields a bitstring for the HF occupation). This state can be used as a starting point before applying the ansatz circuit.
- **QASM Export:** While PennyLane by itself doesn't output QASM directly, you can convert a PennyLane circuit to other frameworks. One strategy is to **convert PennyLane circuits to Qiskit circuits** (PennyLane has a Qiskit device interface). Alternatively, since PennyLane circuits are Pythonic, you can extract a list of gates and translate to QASM manually. For a quick solution, if using PennyLane's Qiskit device: build the circuit, then use Qiskit's `QuantumCircuit.qasm()` as usual. Thus, circuits designed in PennyLane (e.g. variational circuits for H₂, LiH, etc.) are ultimately exportable to QASM2/3 via another library.

Variation: Using PennyLane, you can generate many circuit variations by varying ansatz structure and optimization parameters. For example, you could produce: hardware-efficient ansatz circuits of depth d for multiple molecules; alternative ansatz like **LDCA (Layered Hardware Ansatz)** or custom entangler patterns; or even use PennyLane's *strawberry fields* for quantum optics-style circuits (if relevant, though likely not needed here). PennyLane's strength is in creating differentiable circuits, which might not add new *types* of circuits beyond what Qiskit/OpenFermion offer, but it's another route to ensure diversity in circuit construction.

4. PySCF + Qulacs (Quantum Native Dojo Example)

Qulacs is a fast quantum circuit simulator. While Qulacs itself doesn't interface with PySCF directly, one can use PySCF (via OpenFermion) to get the Hamiltonian and then build circuits in Qulacs. The Quantum Native

Dojo tutorial demonstrates this integration: PySCF computes the H_2 Hamiltonian (using OpenFermion-PySCF) and then a **hardware-efficient ansatz** circuit is built with Qulacs gates ¹³ ⁶ . Notable points:

- The example uses a hardware-efficient ansatz inspired by Kandala *et al.* (2017), consisting of alternating single-qubit rotations (RY, RZ) on each qubit and CZ entangling gates in a chain topology ⁷ ²³ . This is a mainstream variational form for small molecules. The circuit depth is chosen equal to the number of qubits (which was the number of orbitals in the active space) ²⁴ .
- With Qulacs, after defining the ansatz as a `QuantumCircuit` object, one can easily obtain the gate count, depth, etc., and run simulations. To export such circuits to QASM, one would need to manually translate, since Qulacs doesn't natively output QASM. However, because Qulacs circuits can be iterated gate by gate, you could write a small converter. (Alternatively, one could reconstruct the same ansatz in Qiskit using the described gate pattern to then export it.)

The takeaway is that **using PySCF + Qulacs lets you generate circuits with different ansatzes** (the tutorial uses a hardware-efficient ansatz, but one could also implement a UCCSD ansatz in Qulacs if desired). Each ansatz applied to each molecule is a unique circuit. Qulacs focuses on simulation speed, so it's good for verifying circuits, but for QASM output you'd convert through another route.

5. PySCF + MindSpore Quantum (MindQuantum)

MindSpore's **MindQuantum** library (from MindSpore AI framework) also offers quantum chemistry VQE tools that integrate with PySCF:

- MindQuantum uses OpenFermion-PySCF to get molecular data (just like above), and it provides a one-step function `generate_uccsd` which **automatically constructs the UCCSD ansatz circuit** for a given molecule ²⁵ . This function reads the molecule's integrals (optionally the CCSD amplitudes from PySCF) and returns: a parameterized quantum circuit (as a MindQuantum `Circuit` object) for UCCSD, the initial guess parameters (from classical CCSD), and the qubit Hamiltonian ²⁶ ²⁷ . Essentially, MindQuantum is packaging the whole pipeline: "molecule -> UCCSD circuit".
- The output circuit from `generate_uccsd` includes the Hartree-Fock state preparation followed by all the UCCSD excitation gates ²⁸ . The MindQuantum docs show an example for LiH where the UCCSD circuit uses 12 qubits and has 12612 gates (with 44 parameters) ²⁹ ³⁰ – this is a full, unoptimized UCCSD circuit for a modest molecule, which is a realistic benchmark circuit (it's *deep*, but that's common for chemistry ansatzes).
- **QASM Export:** MindQuantum's circuit can be converted to QASM as well, although it's not one-click. One would need to map MindQuantum gates to an OpenQASM representation. If needed, you could translate MindQuantum circuits to Qiskit (since MindQuantum supports a limited QASM import/export internally or via ProjectQ). Given that MindQuantum is less standard, an easier path might be to use the same molecule's data to construct an equivalent UCCSD circuit in Qiskit or Cirq for QASM output. Nonetheless, MindQuantum demonstrates that *many variations of UCC circuits* can be generated easily (different molecules yield different circuit structures and parameter counts).

Variation: Using MindQuantum or similar tools, you can auto-generate circuits for many molecules quickly. For example, one could loop over a list of molecules (H_2 , LiH , BeH_2 , H_2O , NH_3 , CH_4 , etc.), and use `generate_uccsd` to get each molecule's UCCSD circuit. This would yield dozens of circuits in a consistent format. Each of these circuits can then be exported (directly or via translation) to QASM2/3. The advantage is that MindQuantum ensures the ansatz is constructed in a standardized way (including only relevant excitations given the molecule's electrons).

6. Other Frameworks and Considerations

Beyond the above, a few other combinations are possible (though less mainstream, they can still produce valid circuits for your dataset):

- **Tequila or QCOR:** Libraries like Tequila (an open-source quantum chemistry optimization framework) allow users to specify molecules (often via PySCF) and ansatze, then they can output circuits to backend SDKs like Qiskit or Cirq. If available, these could produce additional circuit forms (Tequila, for instance, makes it easy to try adaptive ansatz circuits or different optimization strategies).
- **Quantum Package Integrations:** Some research packages (e.g. *OpenQEMIST* by 1QBit, or *Q# Chemical simulation library*) use PySCF or integrals from PySCF under the hood ³¹. For example, Microsoft's Q# chemistry library used NWChem/PySCF to get Hamiltonians and then constructed Q# circuits for phase estimation. If you need QASM, those Q# circuits could potentially be translated (though this is complex and likely unnecessary for your purposes).
- **QAOA Circuits:** Note that **QAOA** (Quantum Approximate Optimization Algorithm) circuits are generally for combinatorial optimization (e.g. Max-Cut, SAT problems), not directly related to PySCF or chemistry. If by "QAOM" the intention was QAOA, you can certainly include QAOA circuits in your dataset *in addition* to chemistry VQE circuits. QAOA circuits can be generated using, say, Qiskit's QAOA module or other frameworks, by specifying a cost Hamiltonian (e.g. an Ising model of a graph). These circuits consist of alternating cost and mixer unitary layers parameterized by angles. While PySCF plays no role in QAOA, it's straightforward to create, for example, 100 QAOA circuits for 100 random graph instances (each circuit at a fixed depth p). Qiskit or PennyLane can export these to QASM as well. Including QAOA circuits would enhance the diversity of your dataset (since they have different structures than chemistry circuits).
- **Circuit Parameter Variations:** To further enlarge the set of circuits, you might also consider that **different parameter sets yield technically different circuit instances** (though same structure). For example, a UCCSD ansatz with all parameters = 0 versus random parameters – structurally the gates are the same, but if you consider the parameterized gates as part of the circuit data, you could sample different random initializations. However, since your focus is on circuit *structure* for optimization benchmarks, it's probably better to treat a given ansatz + molecule as one circuit (the optimizer will then vary the parameters). Thus, focus on structural variations (different gate arrangements) rather than just different numeric parameters.

7. Summary of Approaches and Mainstream Variations

- **PySCF + Qiskit:** Use PySCF as a driver in Qiskit Nature to get molecular Hamiltonians, then use Qiskit's **Hartree-Fock state + UCCSD ansatz circuits** (or hardware-efficient ansatz). Export via Qiskit to OpenQASM 2 or 3 ⁹. This approach is *very standard* in quantum chemistry VQE research ³, so circuits here are highly legitimate. You can generate many circuits by iterating over molecules and ansatz types.
- **PySCF + OpenFermion (Cirq):** Use PySCF with OpenFermion to get the fermionic Hamiltonian and qubit mapping. Then either:
 - Build **UCCSD circuits** using OpenFermion's excitation generators and implement them in Cirq (or even Qiskit).
 - Build **trotterized evolution circuits** for the Hamiltonian (serves as QPE or VQE trial circuits). For example, one method is the Low-Depth Trotter circuits from OpenFermion's trotter step recipes ³².
- These circuits can be output via Cirq's QASM exporter. They are slightly less common in literature than UCCSD, but still standard methods for simulating chemistry ¹⁴.
- **PySCF + PennyLane:** Use PennyLane's quantum chemistry module with PySCF to get Hamiltonians ²¹. Then construct **variational circuits** using PennyLane templates (hardware-efficient layers, etc.) or custom UCCSD. PennyLane ensures the circuits are differentiable and easy to manipulate. You'd convert these circuits to QASM via Qiskit conversion. This approach is mainstream in that PennyLane is a popular tool for quantum ML and chemistry, and it directly leverages PySCF for correct chemistry data ²¹.
- **PySCF + Qulacs (or others):** Use PySCF (via OpenFermion) to get Hamiltonian, then build circuits in other simulators like Qulacs. The **hardware-efficient VQE ansatz** implemented in the Qulacs tutorial ⁷ is a good example of a standardized circuit (it mimics a known experimental ansatz). You can create similar circuits of various depths and coupling patterns. Since Qulacs circuits can't natively export QASM, you might recreate them in Qiskit for output, but conceptually this gives you more circuit families.
- **PySCF + MindQuantum:** Use PySCF results to directly output **UCCSD circuits including HF state** in MindQuantum ²⁸. This yields a ready-to-go circuit (the MindQuantum summary shows an example with thousands of gates for LiH UCCSD ²⁹, which underscores how complex these circuits can be – useful for optimization benchmarking). These circuits can be translated to QASM with some effort, or you can use them as inspiration to assemble similar circuits in Qiskit/OpenFermion. MindQuantum's approach is quite systematic and ensures you include all singles/doubles excitations from the reference, which is a *legitimate* VQE circuit construction ²⁵.

By mixing and matching the above methods, it's feasible to generate **1,000+ unique circuits**. For example, suppose you take 50 molecules and for each generate: a UCCSD circuit, a hardware-efficient ansatz circuit, and maybe one trotterized simulation circuit – that's 150 circuits. Expand this by considering different basis sets or active space sizes per molecule, or different ansatz depths, and the count grows further. Including some QAOA circuits for variety can add hundreds more (each QAOA instance yields a distinct circuit graph/

structure). All these circuits can be saved as OpenQASM 2.0 or 3.0, which makes them portable across quantum software.

Crucially, each circuit you generate following these mainstream approaches will be **legitimate and representative** – grounded in either a known quantum chemistry ansatz or a well-studied variational form. This ensures your optimization benchmarks use realistic circuits. Below is a quick recap of sources confirming these integration methods:

- *Qiskit Nature with PySCF*: Qiskit's PySCF driver can produce a qubit Hamiltonian for H_2 ³³, and Qiskit provides UCCSD ansatz circuits ready to use ³. These circuits (QuantumCircuit objects) are easily exportable to OpenQASM ⁹.
- *OpenFermion + PySCF*: OpenFermion's integration with PySCF allows constructing qubit Hamiltonians and compiling them to circuits. One can use basis-change or Trotter-step methods to get an optimized quantum circuit from the molecule ¹⁴. The H_2 example resulted in a 4-qubit Cirq circuit after using those methods ¹⁵.
- *PennyLane + PySCF*: PennyLane's documentation shows how to use `method='pyscf'` to build molecular Hamiltonians ²¹. This combines PySCF's accurate integrals with PennyLane's circuit libraries, enabling custom ansatzes for VQE.
- *Qulacs + PySCF*: The Qulacs VQE tutorial uses PySCF (via OpenFermion) to get the H_2 Hamiltonian then builds a hardware-efficient ansatz circuit on Qulacs ⁶. The ansatz (inspired by Kandala's hardware-efficient VQE) is explicitly constructed with RY, RZ rotations and CZ gates ⁷, a standard choice for chemistry benchmarks.
- *MindQuantum + PySCF*: MindQuantum's `generate_uccsd` automates creating a UCCSD circuit (HF state + UCCSD gates) from PySCF outputs ²⁵ ²⁸. The documentation confirms this yields a *complete quantum circuit* for the molecule's ground state ansatz, listing qubit count, gate count, and parameters ²⁹.

Each of these methods will enrich your quantum circuit dataset with **mainstream, diverse circuits** suitable for testing optimization algorithms. By leveraging PySCF in these ways, you ensure the circuits are grounded in real quantum chemistry problems (for VQE) or standard optimization problems (for QAOA), making your ~1000-2000 circuit dataset robust for benchmarking.

Sources:

- PySCF driver integration in Qiskit Nature (obtaining H_2 Hamiltonian) ³³; Qiskit's UCCSD ansatz usage ³. Qiskit circuits export to OpenQASM readily ⁹.
- OpenFermion-PySCF example with Cirq (compiling H_2 to a quantum circuit) ¹⁴ ¹⁵.
- PennyLane documentation on using PySCF as backend for molecular Hamiltonians ²¹.
- Qulacs VQE tutorial using PySCF (OpenFermion) and a hardware-efficient ansatz circuit ⁶ ⁷.
- MindQuantum documentation showing one-click UCCSD circuit generation from PySCF results ²⁵ ²⁸ (yielding an HF+UCCSD circuit for LiH).
- General note on exporting circuits to QASM from any framework (e.g. build in Qiskit then export) ⁹.

1 2 12 33 **Electronic structure - Qiskit Nature 0.7.2**

https://qiskit-community.github.io/qiskit-nature/tutorials/01_electronic_structure.html

3 11 **QiskitSolver - Qiskit Nature PySCF 0.4.0**

https://qiskit-community.github.io/qiskit-nature-pyscf/stubs/qiskit_nature_pyscf.QiskitSolver.html

4 5 **SQD Implementation | IBM Quantum Learning**

<https://quantum.cloud.ibm.com/learning/en/courses/quantum-diagonalization-algorithms/sqd-implementation>

6 7 13 23 24 **6-2. Implementation of variational quantum eigensolver (VQE) using Qulacs — Quantum Native Dojo documentation**

https://dojo.qulacs.org/en/latest/notebooks/6.2_qulacs_VQE.html

8 9 10 **How to create a circuit with multiple registers | Configure circuits | Execute | Fire Opal | Q-CTRL Documentation**

<https://docs.q-ctrl.com/fire-opal/execute/configure-circuits/how-to-create-a-circuit-with-multiple-registers>

14 15 16 32 **Bored with classical computers? – Quantum AI with OpenFermion · Qooba**

<https://blog.qooba.net/2021/08/14/bored-with-classical-computers-quantum-ai-with-openfermion/>

17 **[PDF] OpenFermion: The Electronic Structure Package for Quantum ...**

<https://scispace.com/pdf/openfermion-the-electronic-structure-package-for-quantum-4ue4n3045c.pdf>

18 19 20 21 **Using PennyLane with PySCF and OpenFermion | PennyLane Demos**

https://pennylane.ai/qml/demos/tutorial_qchem_external

22 **Quantum Computing for Quantum Chemistry with qc2**

<https://blog.esciencecenter.nl/quantum-computing-for-quantum-chemistry-with-qc2-69ee18ef2969>

25 26 27 28 29 30 **VQE Application in Quantum Chemistry Computing | MindSpore Quantum 0.10 documentation | MindSpore**

https://www.mindspore.cn/mindquantum/docs/en/r0.10/case_library/vqe_for_quantum_chemistry.html

31 **The Variational Quantum Eigensolver (VQE) - OpenQEMIST - 1QBit**

http://openqemist.1qbit.com/docs/vqe_microsoft_qsharp.html